

作业 1 Unix 进程、文件 和 用户

同济大学计算机系操作系统 国豪 2023 拔尖班 2023

姓名 学号

习题 1: Unix V6++ 进程控制块包括一个 Process 结构和一个 User 结构。抄写 Process、User 结构定义，注释每个字段。酌情加入自己的理解。

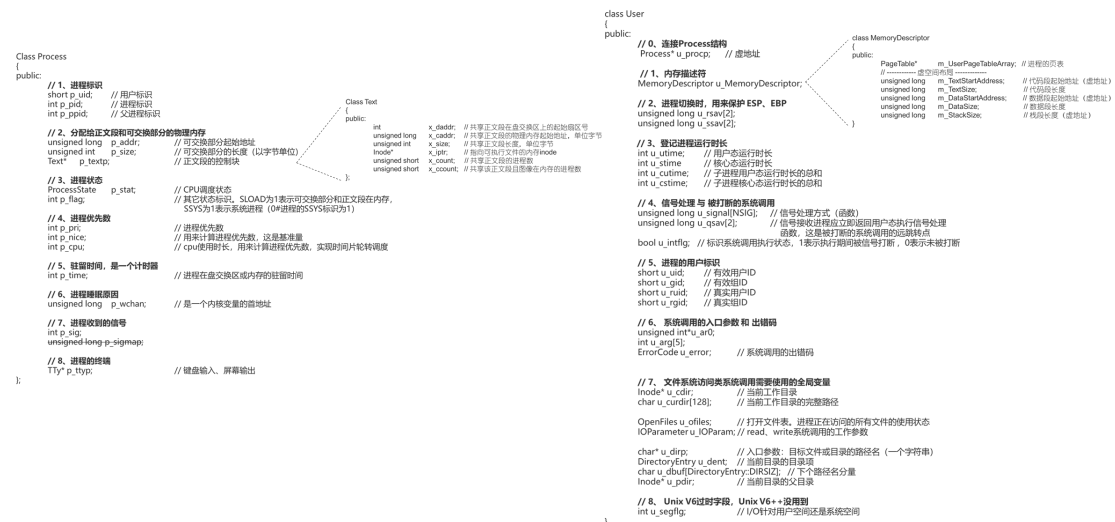


图 1、Unix V6++ 的进程控制块

一、Unix V6++ 进程图像 & 地址

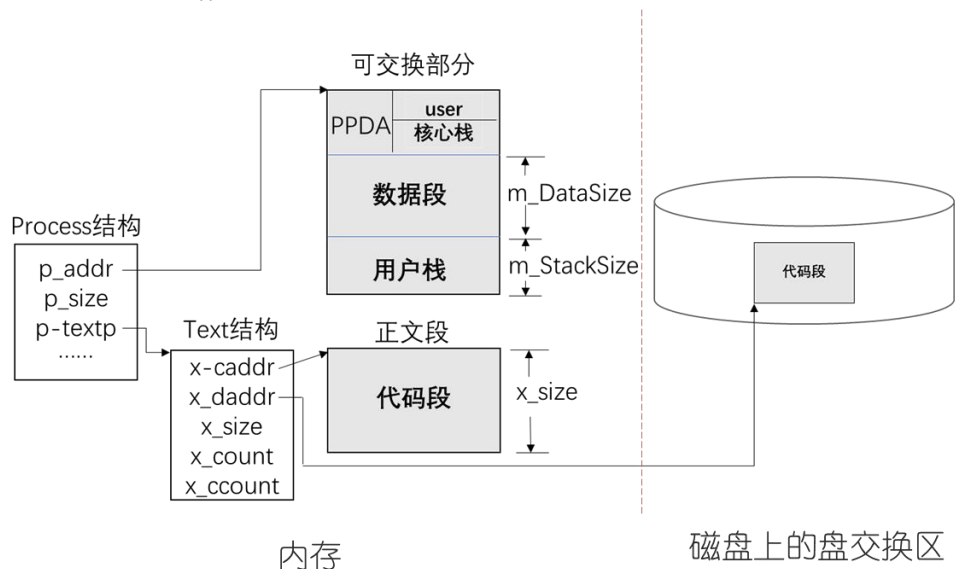


图 2、物理空间中的 Unix V6++ 进程图像



Unix V6++ 进程的虚空间

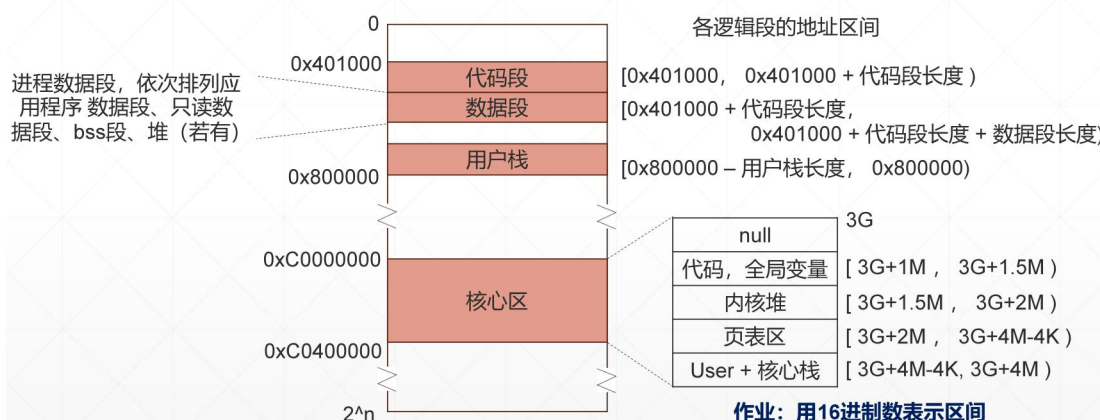


图 3、虚空间中的 Unix V6++ 进程图像

所有逻辑段 4096 字节对齐（首地址是 4096 字节的整数倍，长度是 4096 字节的整数倍）

习题 2：Unix V6++ 系统。进程 PB，3 页代码，1 页数据，1 页 bss，2 页堆栈。问，

(1) 虚空间中，代码段首地址=()，数据段首地址=()，
bss 段首地址=()，用户栈首地址=()。

(2) 物理空间中，

代码段长度 x_size =()，可交换部分长度 p_size =()

已知 $x_caddr = 0x600000$ ， $p_addr = 0x800000$ ，求物理空间中

代码段首地址=()，数据段首地址=()，

bss 段首地址=()，用户栈首地址=()。

(3) 仿照图 2、图 3，画进程 PB 图像，求进程 PB 的页表。

(4) 求 指令、数据地址。

4.1 PB 进程取代码段第 1 条指令。CPU 执行单元给出虚地址=()，MMU 地址映射后生成的物理地址=()。图示 MMU 地址映射过程。

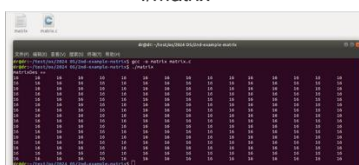
习题 3：Unix V6++ 系统。已知 matrix 进程，代码段 3 页。

$x_caddr = 0x500000$ ， $p_addr = 0x600000$ 。求

```
# include <stdlib.h>
# include <stdio.h>

# define M 16
int version = 1;
int matrixOriginal[M][M];
int matrixDes[M][M];
```

./matrix



```
void produce ( int row, int column )
{
    int i;
    for (i=0;i<M;i++)
        matrixDes[row][column] += matrixOriginal[row][i] * matrixOriginal[i][column];
}

int main()
{
    int i, j;

    for (i=0;i<M;i++)
        for (j=0;j<M;j++)
            produce(i, j);

    printf("matrixDes == \n");
    for (i=0;i<M;i++) {
        for (j=0;j<M;j++)
            printf("%d\t", matrixDes[i][j]);
        printf("\n");
    }
}
```

- (1) 数据段长度=(), 用户栈长度=(),
可交换部分长度=()。
- (2) 选择、填空:
- main 函数存放在()。
 - version 变量是(), 存放在(), 虚地址=(), 物理地址=()。
 - 矩阵 matrixOriginal 是(), 存放在(), 虚地址=(), 物理地址=()。
 - 矩阵 matrixDes 是(), 存放在(), 虚地址=()。 物理地址=()。
 - 矩阵元素 matrixDes[2][2] 虚地址=()。 物理地址=()。
 - 访问寄存器 EAX。CPU 执行单元给出虚地址=(), MMU 地址映射后生成的物理地址=()。
- 红色是单选题。选填 (a) 代码段 (b) 数据段 (c) bss 段 (d) 堆栈段 (e) 带初值的全局变量 (f) 不带初值的全局变量 (g) 局部变量。
黑色是填空题。

二、进程 uid 和 文件 uid

习题 4: 某 Linux 系统, 10 台终端。花名册文件内容如下:

user name	password	uid	gid	PATH	initial working directory	initial shell
root	X	0	0	*****	/root	/bin/sh
.....						
John	X	12	1	*****	/home/John	/bin/sh
Mary	X	13	1	*****	/home/Mary	/bin/sh
.....						
Tony	X	58	1	*****	/home/Tony	/bin/x_windows

T 时刻, Tony 用户使用 7#终端, Mary 用户使用 5#终端, 如图:

Login: Tony
Password: *****
没碰过cd.....
\$ gcc -o Matrix Matrix.c

用 7# 终端

Login: Mary
Password: *****
没碰过cd.....
\$ gcc -o HelloWorld HelloWorld.c

用 5# 终端

- (1) 在目录树上标出可执行文件 Matrix 和 HelloWorld 的位置, 确定可执行文件 Matrix 的绝对路径名和 uid, 确定可执行文件 HelloWorld 的绝对路径名和 uid,
- (2) 这 2 个 gcc 进程, p_uid 和 u_cdir? p_ttyp?
- (3) 一周后, Tony 重新编译 Matrix.c 程序。为之服务的进程, p_uid 和 u_cdir? p_ttyp?

三、进程打开文件结构 和 读写操作

习题 4：画语句 1、2、3、4 执行结束后，example2 进程的打开文件结构

```
int main() // example2.c
{
    int fd1, fd2;
    char c;

    1. fd1 = Open("foobar.txt", O_RDONLY, 0);
    2. fd2 = Open("foobar.txt", O_RDONLY, 0);
    3. Read(fd1, &c, 1);
    4. Read(fd2, &c, 1);
    printf("c = %c\n", c);
    exit(0);
}
```

习题 5：磁盘文件 file1，长度是 700 个字节。0~99 字节是 100 个字符 A，100~199 字节是 100 个字符 B，200~299 字节是 100 个字符 C……。

```
int fd = open(file1, RDWR); //读写打开
char data[300];
lseek(fd, 500, 0);
int count = read (fd, data, 300);
count = write(fd, data, 300);
```

问：这段程序执行完毕后，文件 file1 的长度是()，文件的内容？

四、fork 系统调用

例题 1：下面的这个程序会输出几个整数？请写出程序的输出，并请在代码中标出父进程执行的所有语句和子进程执行的所有语句。

```
L1: #include <stdio.h>
L2: void main (void )
L3: {
L4:     int i = 10, x ;
L5:     if ( x = fork() )
L6:     {
L7:         i += 10;
L8:         printf ( "%d\t", i );
L9:     }
L10:    else
L11:        printf ( "%d\t", i );
L12:    printf ( "%d\t", i );
L13:}
```

参考答案：这个程序会输出 4 个整数。这是因为语句 L12 执行了 2 次。

程序的输出：

可能是：	10	20	10	20
可能是：	10	20	20	10
还可能是：	20	10	20	10
或：	20	10	10	20
或：	20	20	10	10
或：	10	10	20	20

极端的情况下，还会出现下面的结果（为什么？原因我忘了）：

```

L1: #include <stdio.h>
L2: void main (void )
L3: {
L4:     int i = 10, x; //父
L5:     if ( x = fork() ) //父, 子
L6:     {
L7:         i += 10; // 父
L8:         printf ( "%d\t", i ); // 父
L9:     }
L10:    else
L11:        printf ( "%d\t", i ); //子
L12:    printf ( "%d\t", i ); //父, 子
L13:}

```

例题 2：阅读下面这段代码。父进程 P0 从 main()入口开始跑，先后创建 2 个子进程 P1、P2。请在代码中标注子进程 P1、P2 执行的代码片段。

```

int main( )
{
    .....
    int pid = fork ( ); // P0
    if (pid == 0)
        代码片段 1;
    else {
        pid = fork ( ); // P0
        if (pid == 0)
            代码片段 2;
        else
            代码片段 3; // P0
    }
}

```